

BÀI 7. QUẢN LÝ SCHEMA VỚI MIGRATIONS

Bài trước, chúng ta đã kết nối thành công ứng dụng với CSDL. Tuy nhiên, cách chúng ta để EF Core tự tạo CSDL là một “phép thuật” chỉ dùng được một lần. Chuyện gì sẽ xảy ra khi bạn cần thêm một cột mới vào bảng sản phẩm? Xóa CSDL và tạo lại? Đó là không thể chấp nhận được trong thế giới thực. Bài này, chúng ta sẽ học cách quản lý CSDL của mình bằng **Migrations**.

MỤC TIÊU BÀI HỌC

Sau khi hoàn thành Bài học này, bạn sẽ có khả năng:

- Giải thích được tại sao `Database.EnsureCreated()` là một phương pháp nguy hiểm và chỉ nên dùng cho mục đích thử nghiệm.
- Sử dụng thành thạo quy trình làm việc với EF Core Migrations: từ việc thay đổi Model đến việc tạo và áp dụng migration.
- Hiểu được một file migration, hiểu rõ vai trò của phương thức `Up()` và `Down()`.
- Triển khai một quy trình seeding data (điền dữ liệu mẫu) mạnh mẽ và an toàn, đảm bảo ứng dụng luôn có dữ liệu để hoạt động.

NỘI DUNG CHI TIẾT

7.1. Giới thiệu EF Core Migrations – “Git” cho Cơ sở dữ liệu

7.2. Vấn đề của `Database.EnsureCreated()`

Ở buổi trước, EF Core đã ngầm sử dụng một cơ chế tương tự `Database.EnsureCreated()`.

- **Nó làm gì?** Nó kiểm tra xem CSDL có tồn tại không. Nếu không, nó tạo ra CSDL và schema dựa trên model hiện tại.
- **Vấn đề:** Nếu CSDL đã tồn tại, nó sẽ **không làm gì cả**. Điều này có nghĩa là nếu bạn thêm một thuộc tính mới vào class `Product`, cột tương ứng sẽ **không bao giờ** được thêm vào bảng `Products` đã có. Lựa chọn duy nhất của bạn là xóa CSDL đi để nó tạo lại, dẫn đến **mất toàn bộ dữ liệu hiện có**.

7.2.1. Migrations: “Cỗ máy thời gian” cho CSDL

Hãy tưởng tượng **Migrations** giống như hệ thống quản lý phiên bản **Git** mà các lập trình viên dùng cho code.

- Model của bạn là code.
- Schema của CSDL là “kết quả build” của code đó.
- Một Migration chính là một “commit” cho schema CSDL.

Nó không lưu lại toàn bộ CSDL, mà nó ghi lại **sự thay đổi** từ phiên bản này sang phiên bản tiếp theo. Mỗi migration là một file C# chứa hai bộ "chỉ dẫn":

1. **Up()**: Hướng dẫn cách **nâng cấp** CSDL lên phiên bản mới (ví dụ: CREATE TABLE, ADD COLUMN).
2. **Down()**: Hướng dẫn cách **hạ cấp** (hoàn tác) về phiên bản cũ (ví dụ: DROP TABLE, DROP COLUMN).

7.2.2. Quy trình làm việc chuẩn

Đây là chu trình bạn sẽ lặp đi lặp lại trong suốt quá trình phát triển dự án:

1. **Thay đổi Model**: Bạn chỉnh sửa các class Entity trong C# (ví dụ: thêm một thuộc tính public string? Color { get; set; } vào Product.cs).
2. **Tạo Migration (Add-Migration)**: Bạn ra lệnh cho EF Core. Nó sẽ hành động như một “thám tử”, so sánh model C# hiện tại của bạn với “ảnh chụp” (snapshot) của CSDL ở lần migration trước, tìm ra sự khác biệt và tự động viết một file migration mới mô tả chính xác sự thay đổi đó.
3. **Áp dụng Migration (Update-Database)**: Bạn ra lệnh cho EF Core. Nó sẽ đọc file migration mới nhất và thực thi các chỉ dẫn trong phương thức Up() để cập nhật CSDL thực tế.

7.3. Thực thi Migrations - Biến Code thành Cấu trúc CSDL

7.3.1. Lệnh Add-Migration <TênMigration>

- **Bạn chạy lệnh**: Add-Migration AddProductColor
- **EF Core làm gì ở hậu trường?**
 1. Nó nhìn vào các class Model của bạn (Product.cs, etc.).

2. Nó so sánh chúng với file YourDbContextModelSnapshot.cs. File này là “bức ảnh” về cấu trúc CSDL mà EF Core nghĩ là đang tồn tại, dựa trên migration cuối cùng.
3. Nó thấy: class Product có thêm thuộc tính Color mà trong snapshot không có.
4. Nó tự động tạo ra một file C# mới (ví dụ 20251015100000_AddProductColor.cs) với nội dung:

```
//C#
public partial class AddProductColor : Migration
{
    // Hướng dẫn để đi tới tương lai (NÂNG CẤP)
    protected override void Up(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.AddColumn<string>(
            name: "Color",
            table: "Products",
            type: "nvarchar(max)",
            nullable: true); // Cho phép giá trị null
    }

    // Hướng dẫn để quay về quá khứ (HẠ CẤP)
    protected override void Down(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.DropColumn(
            name: "Color",
            table: "Products");
    }
}
```

5. Nó cập nhật lại file YourDbContextModelSnapshot.cs để “bức ảnh” mới bao gồm cả cột Color.

7.3.2. Lệnh Update-Database

- **Bạn chạy lệnh:** Update-Database
- **EF Core làm gì?**

1. Nó kết nối tới CSDL của bạn.
2. Nó kiểm tra một bảng đặc biệt tên là **__EFMigrationsHistory**.

3. Bảng này giống như một **cuốn sổ nhật ký**, ghi lại tên của tất cả các file migration đã được "thi công" trước đây.
4. Nó so sánh danh sách migration trong code với danh sách trong sổ nhật ký.
5. Nó thấy: "Aha, migration 20251015100000_AddProductColor có trong code nhưng chưa có trong sổ nhật ký."
6. Nó sẽ chạy phương thức Up() của migration đó, thực thi câu lệnh ALTER TABLE Products ADD Color nvarchar(max) NULL.
7. Sau khi chạy thành công, nó ghi tên migration 20251015100000_AddProductColor vào sổ nhật ký __EFMigrationsHistory để đảm bảo nó không bao giờ chạy lại lần nữa.

7.4. Seeding Data – “Gieo mầm” dữ liệu cho ứng dụng

7.4.1. Tại sao cần Seeding?

Một CSDL có cấu trúc hoàn hảo nhưng trống rỗng thì không thể dùng để phát triển hay kiểm thử được. Seeding Data là quá trình **tự động điền dữ liệu ban đầu** vào CSDL.

- **Dữ liệu cho Development/Testing:** Cần có một vài sản phẩm mẫu để hiển thị trên trang chủ.
- **Dữ liệu nền tảng:** Cần có sẵn một tài khoản Admin và vai trò Admin để có thể đăng nhập vào trang quản trị.
- **Dữ liệu mặc định:** Cần có sẵn danh sách các quốc gia, các loại danh mục...

7.4.2. Cách triển khai: An toàn và Tự động

Chúng ta sẽ tạo một phương thức tĩnh, phương thức này sẽ được gọi mỗi khi ứng dụng khởi động.

– Quy trình trong lớp SeedData.cs:

1. Lấy một instance của DbContext từ DI Container.
2. **(Rất quan trọng cho triển khai)** Tự động áp dụng các migration đang chờ: context.Database.Migrate();. Dòng này đảm bảo rằng trước khi seed data, cấu trúc CSDL đã ở phiên bản mới nhất.
3. Kiểm tra xem dữ liệu đã tồn tại hay chưa: if (!context.Products.Any()). Đây là

chốt chặn an toàn để tránh việc thêm dữ liệu trùng lặp mỗi lần khởi động lại ứng dụng.

4. Nếu chưa có dữ liệu, thêm dữ liệu vào DbSet.
5. Gọi `context.SaveChanges()` để lưu dữ liệu xuống CSDL.

– **Gọi từ Program.cs:**

- Chúng ta sẽ gọi phương thức seeding này ở cuối file `Program.cs`, ngay trước `app.Run()`. Điều này đảm bảo tất cả các dịch vụ đã được cấu hình và sẵn sàng để sử dụng.

7.5. Hướng dẫn thực hành

7.5.1. Bước 0: Dọn dẹp môi trường cũ

Để thấy sức mạnh của Migrations, chúng ta cần xóa CSDL cũ đã được tạo tự động.

1. Dùng **SQL Server Object Explorer** trong Visual Studio, tìm đến CSDL `SportsStore` và xóa nó đi.
2. **Hoặc** dùng File Explorer, vào thư mục người dùng (`C:\Users\YourUsername`) và xóa 2 file `SportsStore.mdf` và `SportsStore_log.ldf`.

7.5.2. Bước 1: Tạo Migration đầu tiên

Cách 1: Visual Studio (Package Manager Console - PMC)

1. Mở PMC: Tools -> NuGet Package Manager -> Package Manager Console.
2. Đảm bảo Default project được chọn là `SportsStore.Infrastructure` (vì `DbContext` nằm ở đây).
3. Chạy lệnh:

```
//PowerShell
# Add-Migration: Lệnh để tạo migration.
# InitialCreate: Tên bạn đặt cho migration đầu tiên, mang ý nghĩa "Khởi tạo ban đầu".
Add-Migration InitialCreate
```

Cách 2: VS Code + .NET CLI

1. Mở terminal tại thư mục gốc của **solution** (thư mục chứa file `.sln`).

2. Chạy lệnh:

```
//Bash
# dotnet ef: Công cụ dòng lệnh của Entity Framework.
# migrations add: Lệnh con để tạo migration.
# InitialCreate: Tên của migration.
# --project: Cờ này BẮT BUỘC, chỉ định project nào chứa DbContext.
dotnet ef migrations add InitialCreate --project SportsStore.Infrastructure
```

Kết quả: Một thư mục Migrations sẽ được tạo trong project Infrastructure, chứa file migration mới.

7.5.3. Bước 2: Áp dụng Migration để tạo CSDL

Cách 1: Visual Studio (PMC)

```
//PowerShell
# Update-Database: Lệnh để áp dụng tất cả các migration chưa được thi công.
Update-Database
```

Cách 2: VS Code + .NET CLI

```
//Bash
# dotnet ef database update: Lệnh con để cập nhật CSDL.
dotnet ef database update --project SportsStore.Infrastructure
```

Kết quả: CSDL SportsStore và các bảng (Products, __EFMigrationsHistory) sẽ được tạo ra.

7.5.4. Bước 3: Seed dữ liệu

1. Tạo lớp SeedData.cs trong SportsStore.Infrastructure.

```
//C#
// SportsStore.Infrastructure/SeedData.cs
using Microsoft.AspNetCore.Builder;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using SportsStore.Domain;

public static class SeedData {
    public static void EnsurePopulated(IApplicationBuilder app) {
        // Lấy DbContext từ DI Container. Đây là cách chuẩn để lấy dịch vụ
```

```
// khi bạn không ở trong một class được DI hỗ trợ trực tiếp.
ApplicationDbContext context = app.ApplicationServices

.CreateScope().ServiceProvider.GetRequiredService<ApplicationDbContext>();

// Tự động chạy migration nếu có
if (context.Database.GetPendingMigrations().Any()) {
    context.Database.Migrate();
}

// Chốt chặn an toàn: chỉ thêm dữ liệu nếu bảng trống
if (!context.Products.Any()) {
    // AddRange cho phép thêm nhiều đối tượng cùng lúc, hiệu quả
    // hơn Add từng cái.
    context.Products.AddRange(
        new Product { Name = "Thuyền Kayak", Description = "Chiếc
        thuyền nhỏ cho một người chèo.", Category = "Thể thao dưới nước", Price =
        6600000m },
        new Product { Name = "Áo phao cứu sinh", Description = "Bảo
        hộ an toàn, kiểu dáng thời trang.", Category = "Thể thao dưới nước", Price
        = 1175000m },
        new Product { Name = "Bóng đá tiêu chuẩn", Description =
        "Bóng đạt chuẩn kích thước và trọng lượng FIFA.", Category = "Bóng đá",
        Price = 470000m },
        new Product { Name = "Cờ góc sân", Description = "Tạo vẻ
        chuyên nghiệp cho sân bóng của bạn.", Category = "Bóng đá", Price = 840000m
        },
        new Product { Name = "Mô hình sân vận động", Description =
        "Sân vận động 35,000 chỗ ngồi (đóng gói phẳng).", Category = "Bóng đá",
        Price = 1908000000m },
        new Product { Name = "Mũ Tư Duy", Description = "Cải thiện
        75% hiệu suất hoạt động của não.", Category = "Cờ Vua", Price = 385000m },
        new Product { Name = "Ghế không vững", Description = "Bí
        mật gây bất lợi cho đối thủ của bạn.", Category = "Cờ Vua", Price = 720000m
        },
        new Product { Name = "Bàn cờ người", Description = "Trò
        chơi thú vị cho cả gia đình.", Category = "Cờ Vua", Price = 1800000m },
        new Product { Name = "Quân Vua Kim Cương", Description =
        "Quân Vua mạ vàng, đính kim cương.", Category = "Cờ Vua", Price = 28800000m
        },
        new Product { Name = "Giày chạy bộ", Description = "Nhẹ và
        thoải mái cho quãng đường dài.", Category = "Chạy bộ", Price = 2400000m },
        new Product { Name = "Thảm Yoga cao cấp", Description = "Bề
```

```
mặt chống trượt giúp giữ thăng bằng hoàn hảo.", Category = "Fitness", Price = 840000m },
        new Product { Name = "Bình nước giữ nhiệt", Description = "Giữ lạnh đồ uống trong 24 giờ.", Category = "Fitness", Price = 380000m }
    };
    // Quan trọng: Mọi thay đổi (Add, Update, Remove) chỉ nằm trong bộ nhớ
    // cho đến khi bạn gọi SaveChanges(). Lệnh này sẽ tạo và gửi các câu lệnh INSERT tương ứng đến CSDL.
    context.SaveChanges();
}
}
```

2. Gọi SeedData từ Program.cs trong SportsStore.WebUI.

```
C#
// ...
app.MapControllerRoute("default",
"{controller=Home}/{action=Index}/{id?}");
// Gọi phương thức seeding. "app" được truyền vào để phương thức có thể truy cập vào DI container của ứng dụng.
SeedData.EnsurePopulated(app);

app.Run();
```

CÂU HỎI LÝ THUYẾT

1. Hãy mô tả bằng lời của bạn, tại sao việc xóa và tạo lại CSDL mỗi khi thay đổi model là một ý tưởng tồi trong thực tế?
2. Trong một file migration, phương thức Up() và Down() có vai trò gì?
3. Bảng __EFMigrationsHistory dùng để làm gì? Điều gì sẽ xảy ra nếu bạn xóa bảng này đi?
4. Trong lớp SeedData, tại sao câu lệnh if (!context.Products.Any()) lại quan trọng?

TỔNG KẾT BÀI HỌC

Bài này, chúng ta đã trang bị một trong những kỹ năng quan trọng nhất cho một lập trình viên .NET backend:

- Chúng ta đã làm chủ được **EF Core Migrations**, một công cụ chuyên nghiệp để quản lý và phiên bản hóa cấu trúc CSDL song song với code.

- Chúng ta đã hiểu rõ quy trình làm việc chuẩn: **Thay đổi -> Tạo Migration -> Áp dụng**.
- Chúng ta đã học cách "**gieo mầm**" dữ liệu một cách an toàn, đảm bảo ứng dụng luôn có dữ liệu cần thiết để khởi chạy trên bất kỳ môi trường nào.

Từ giờ, bạn không còn phải lo sợ mỗi khi cần thay đổi CSDL nữa. Bạn đã có trong tay một "cỗ máy thời gian" mạnh mẽ và đáng tin cậy.

BÀI TẬP VỀ NHÀ

1. **Thực hành lặp lại:** Xóa CSDL và thư mục Migrations của bạn. Sau đó, thực hiện lại toàn bộ quy trình: Add-Migration, Update-Database, và chạy ứng dụng để kiểm tra dữ liệu seed.
2. Bài tập kinh điển - Thêm một cột mới:
 - **Bước 1:** Mở file Product.cs và thêm một thuộc tính mới: public string? Manufacturer { get; set; }.
 - **Bước 2:** Chạy lệnh Add-Migration AddProductManufacturer. Mở file migration mới được tạo ra để xem EF Core đã tự động viết code AddColumn như thế nào.
 - **Bước 3:** Chạy lệnh Update-Database.
 - **Bước 4:** Dùng **SQL Server Object Explorer** để kiểm tra xem bảng Products bây giờ đã có thêm cột Manufacturer hay chưa. Bạn đã cập nhật CSDL thành công mà không làm mất dữ liệu!

TÀI LIỆU THAM KHẢO

- [1]. **EF Core Migrations Overview:** [Microsoft Docs - Migrations Overview](#)
- [2]. **Managing Migrations:** [Microsoft Docs - Managing Migrations](#)
- [3]. **Data Seeding:** [Microsoft Docs - Data Seeding](#)